

To create a dynamic breadcrumb component in Next.js using the **App Router**, you can build a reusable **Client Component**. This component automatically parses the URL pathname, splits it into separate navigation tracks, and styles the active route.  Medium · Kristian Cabading +2

Implementation Guide

1. Create the Component

Create a new file called `Breadcrumbs.tsx` in your components directory. This script utilizes Tailwind CSS for fast, responsive utility styling.  Medium +4

tsx

```
'use client';

import React from 'react';
import { usePathname } from 'next/navigation';
import Link from 'next/link';

interface BreadcrumbProps {
  homeLabel?: string;
  separator?: React.ReactNode;
}

export default function Breadcrumbs({ homeLabel = 'Home', separator = '/' }: BreadcrumbPro
  const pathname = usePathname();

  // Split the pathname into segments and remove empty strings
  const pathSegments = pathname.split('/').filter((segment) => segment !== '');

  return (
    <nav aria-label="Breadcrumb" className="my-4">
      <ol className="flex items-center space-x-2 text-sm text-gray-600">
        { /* Root Home Link */ }
        <li>
          <Link href="/" className="hover:text-blue-600 transition-colors">
            {homeLabel}
          </Link>
        </li>

        {pathSegments.map((segment, index) => {
          // Generate the nested route URL for each crumb
```

```

const href = `/${pathSegments.slice(0, index + 1).join('/')}`;
const isLast = index === pathSegments.length - 1;

// Humanize the text segment (e.g., "blog-posts" -> "Blog Posts")
const label = decodeURIComponent(segment)
  .replace(/[-_]/g, ' ')
  .replace(/\b\w/g, (char) => char.toUpperCase());

return (
  <React.Fragment key={href}>
    {/* Visual Separator */}
    <li className="text-gray-400 select-none">{separator}</li>

    {/* Crumb Item */}
    <li>
      {isLast ? (
        <span className="font-semibold text-gray-900" aria-current="page">
          {label}
        </span>
      ) : (
        <Link href={href} className="hover:text-blue-600 transition-colors">
          {label}
        </Link>
      )}
    </li>
  </React.Fragment>
);
  }}}
</ol>
</nav>
);
}

```

Use code with caution.

2. Implement the Component

Import and render your breadcrumbs directly inside a layout or a specific page component.  Medium

tsx

```
import Breadcrumbs from '@components/Breadcrumbs';

export default function Layout({ children }: { children: React.ReactNode }) {
  return (
    <div className="max-w-7xl mx-auto p-4">
      <Breadcrumbs separator={<span className="text-xs">→</span>} />
      <main>{children}</main>
    </div>
  );
}
```

Use code with caution.

Technical Checklist & Best Practices

- **Dynamic Route IDs:** If your URL contains database tracking IDs (e.g., `/products/p9823`), the text string will read "P9823". To replace IDs with actual names, you can build a wrapper component that maps specific key parameters to real titles.  Medium +2
- **Layout Isolation:** Keep `'use client'` inside your breadcrumb file. Importing this small client wrapper into your global server `layout.tsx` guarantees your entire page architecture doesn't accidentally opt out of **Server Side Rendering (SSR)**.  Medium +3
- **Accessibility:** Ensure your markup relies on a `<nav aria-label="Breadcrumb">` wrapper and sets `aria-current="page"` explicitly on the final structural node to meet full [WAI-ARIA accessibility guidelines](#). 
- **Pre-made Alternatives:** If you do not want to manage custom configurations from scratch, you can deploy pre-packaged layouts directly using the [Shadcn UI Breadcrumb component](#) or the atomic configurations from the [Radix UI primitives ecosystem](#). 

If you are dealing with deeply nested dynamics, let me know:

- Are you handling **dynamic route variables** like `[id]` or `[slug]`?
- Are you using **Route Groups** (folders wrapped in parentheses like `(dashboard)`) that should be omitted from the path?  Next.js +3

I can tailor a specific regex transformer to strip out or replace those segments flawlessly.  NPM +1